

01_schema-ingest-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ schemas/  
            └─ ingest.py
```

El archivo `ingest.py` se encuentra dentro de la carpeta de esquemas del backend:

```
backend/app/schemas/
```

Este archivo define el schema `IngestPayload`, que representa la estructura de datos que debe recibir el endpoint de ingesta. Su función principal es validar los datos que llegan a:

```
POST /ingest
```

Antes de que el backend cree un evento en la base de datos, FastAPI y Pydantic comprueban que el JSON recibido cumple las reglas definidas en este schema.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' backend/app/schemas/ingest.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
backend/app/schemas/ingest.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from pydantic import BaseModel, Field  
from typing import Optional, Any
```

```
class IngestPayload(BaseModel):
    source: str = Field(min_length=1, max_length=64)
    severity: int = Field(ge=0, le=10)
    message: str = Field(min_length=1)

    # opcional: para futuro (IP, host, raw, etc.)
    meta: Optional[dict[str, Any]] = None
```

4 Función general del archivo

El archivo `schemas/ingest.py` define la estructura del payload de entrada para la API de ingesta.

La clase principal es:

```
class IngestPayload(BaseModel):
```

Esta clase hereda de `BaseModel`, que pertenece a Pydantic.

Pydantic permite definir modelos de datos con validaciones automáticas.

En este caso, `IngestPayload` indica que un evento recibido por `/ingest` debe tener estos campos:

```
source    → origen del evento
severity  → severidad numérica
message   → mensaje descriptivo
meta      → metadatos opcionales
```

Este schema se utiliza en `backend/app/api/routes/ingest.py`:

```
def ingest(payload: IngestPayload, db: Session = Depends(get_db)):
```

Esto significa que FastAPI validará automáticamente el cuerpo de la petición antes de ejecutar la lógica interna del endpoint.

Si el JSON recibido no cumple el schema, FastAPI devuelve un error de validación y no llega a crear el evento.

5 Estructura general del archivo

El archivo puede dividirse en tres bloques:

```
from pydantic import BaseModel, Field
```

Importación de Pydantic.

```
from typing import Optional, Any
```

Importación de tipos para campos opcionales y valores flexibles.

```
class IngestPayload(BaseModel):
    ...
```

Definición del schema de entrada.

Visualmente:

```
ingest.py
├─ Importaciones Pydantic
├─ Importaciones typing
├─ IngestPayload
│   ├── source
│   ├── severity
│   ├── message
│   └─ meta
```

6 Análisis línea por línea

Importación de BaseModel y Field

```
from pydantic import BaseModel, Field
```

Esta línea importa dos elementos desde Pydantic:

```
BaseModel  
Field
```

BaseModel

`BaseModel` es la clase base de Pydantic.

Cuando una clase hereda de `BaseModel`, Pydantic puede:

- Validar datos de entrada.
- Comprobar tipos.
- Aplicar restricciones.
- Convertir datos si es posible.
- Generar errores claros si algo no cumple el schema.

En este archivo se usa aquí:

```
class IngestPayload(BaseModel):
```

Esto convierte `IngestPayload` en un modelo de validación.

Field

`Field` permite añadir restricciones y metadatos a los campos.

En este archivo se usa en:

```
source: str = Field(min_length=1, max_length=64)  
severity: int = Field(ge=0, le=10)  
message: str = Field(min_length=1)
```

Es decir, `Field` no solo indica que existe un campo, sino también qué condiciones debe cumplir.

Importación de Optional y Any

```
from typing import Optional, Any
```

Esta línea importa dos tipos desde el módulo estándar `typing`.

Optional

`Optional` indica que un valor puede ser del tipo indicado o `None`.

En este archivo se usa aquí:

```
meta: Optional[dict[str, Any]] = None
```

Esto significa que `meta` puede ser un diccionario o puede no enviarse.

Any

Any representa cualquier tipo de dato.

En este archivo se usa dentro de:

```
dict[str, Any]
```

Esto indica que el diccionario `meta` tendrá claves de tipo `str`, pero sus valores pueden ser de cualquier tipo.

Por ejemplo:

```
{
  "host": "server-01",
  "ip": "192.168.1.10",
  "attempts": 5,
  "blocked": true
}
```

Aquí hay valores de tipo texto, número y booleano.

Definición de la clase `IngestPayload`

```
class IngestPayload(BaseModel):
```

Esta línea define la clase `IngestPayload`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
IngestPayload
```

Nombre de la clase.

El nombre indica que representa el payload de ingesta.

```
(BaseModel)
```

Indica que la clase hereda de `BaseModel`.

Esto hace que Pydantic valide automáticamente los datos recibidos.

```
:
```

Marca el inicio del bloque de la clase.

Campo `source`

```
source: str = Field(min_length=1, max_length=64)
```

Esta línea define el campo `source`.

Representa el origen del evento.

Ejemplos posibles:

```
auth
firewall
linux
```

```
windows
ids
web
```

Desglose:

```
source
```

Nombre del campo.

```
: str
```

Indica que debe ser una cadena de texto.

```
= Field(...)
```

Define restricciones adicionales mediante Pydantic.

Restricción `min_length=1`

```
min_length=1
```

Indica que el campo debe tener al menos 1 carácter.

Esto evita aceptar un origen vacío.

Ejemplo inválido:

```
{
    "source": ""
}
```

Restricción `max_length=64`

```
max_length=64
```

Indica que el campo no puede superar los 64 caracteres.

Esto está alineado con el modelo SQLAlchemy `Event`, donde el campo `source` está definido como:

```
source: Mapped[str] = mapped_column(String(64), nullable=False)
```

Por tanto, el schema valida antes de llegar a la base de datos.

Esto evita que PostgreSQL rechace posteriormente un valor demasiado largo.

Campo `severity`

```
severity: int = Field(ge=0, le=10)
```

Esta línea define el campo `severity`.

Representa la severidad del evento.

Desglose:

```
severity
```

Nombre del campo.

```
: int
```

Indica que debe ser un número entero.

```
= Field(ge=0, le=10)
```

Define el rango permitido.

Restricción `ge=0`

```
ge=0
```

`ge` significa:

```
greater or equal
```

Es decir:

```
mayor o igual que 0
```

La severidad mínima aceptada será 0.

Restricción `le=10`

```
le=10
```

`le` significa:

```
less or equal
```

Es decir:

```
menor o igual que 10
```

La severidad máxima aceptada será 10.

Ejemplos:

```
severity = 0    → válido  
severity = 5    → válido  
severity = 10   → válido  
severity = -1   → inválido  
severity = 11   → inválido
```

Esta validación evita recibir severidades fuera de escala.

Campo `message`

```
message: str = Field(min_length=1)
```

Esta línea define el campo `message`.

Representa el mensaje descriptivo del evento.

Desglose:

```
message
```

Nombre del campo.

```
: str
```

Indica que debe ser una cadena de texto.

```
= Field(min_length=1)
```

Exige que tenga al menos un carácter.

Ejemplo válido:

```
{
  "message": "Failed login attempt for user admin"
}
```

Ejemplo inválido:

```
{
  "message": ""
}
```

Este campo también está alineado con el modelo `Event`, donde `message` está definido como obligatorio:

```
message: Mapped[str] = mapped_column(Text, nullable=False)
```

Comentario sobre `meta`

```
# opcional: para futuro (IP, host, raw, etc.)
```

Esta línea es un comentario.

Explica que el campo `meta` es opcional y está pensado para almacenar información adicional del evento.

Ejemplos de datos futuros o complementarios:

```
IP
host
raw
usuario
acción
puerto
servicio
```

Este comentario ayuda a entender por qué `meta` es flexible.

Campo `meta`

```
meta: Optional[dict[str, Any]] = None
```

Esta línea define el campo `meta`.

Representa metadatos opcionales del evento.

Desglose:

```
meta
```

Nombre del campo.

```
: Optional[dict[str, Any]]
```

Indica que el valor puede ser:

- Un diccionario.
- None.

El diccionario debe tener:

```
claves → str  
valores → cualquier tipo
```

-

Indica que el valor por defecto es `None`.

Por tanto, el cliente puede enviar un evento sin `meta`.

Ejemplo sin `meta`:

```
{  
  "source": "auth",  
  "severity": 4,  
  "message": "Failed login attempt"  
}
```

Ejemplo con `meta`:

```
{  
  "source": "auth",  
  "severity": 4,  
  "message": "Failed login attempt",  
  "meta": {  
    "host": "server-01",  
    "user": "admin",  
    "ip": "192.168.1.10"  
  }  
}
```

Resultado final del archivo

Después de cargar este archivo, queda disponible el schema:

```
IngestPayload
```

Este schema define qué datos acepta el endpoint:

```
POST /ingest
```

Campos aceptados:

```
source  
severity  
message  
meta
```

Validaciones aplicadas:

```
source → string entre 1 y 64 caracteres  
severity → entero entre 0 y 10
```

```
message → string mínimo 1 carácter
meta    → diccionario opcional
```

7 Relación con el flujo técnico del laboratorio

IngestPayload participa al inicio del flujo de ingesta.

La relación técnica sería:

```
Cliente envía JSON
    ↓
FastAPI recibe POST /ingest
    ↓
Pydantic valida con IngestPayload
    ↓
si es válido, entra en la función ingest()
    ↓
se crea un Event
    ↓
se guarda en PostgreSQL
    ↓
se evalúan reglas
    ↓
pueden generarse alertas
```

Si el payload no cumple el schema, el flujo se detiene antes de llegar a la base de datos.

Esto protege el backend de datos incompletos o mal formados.

8 Errores típicos o puntos importantes

Falta un campo obligatorio

Los campos obligatorios son:

```
source
severity
message
```

Si falta alguno, FastAPI devolverá un error de validación.

Ejemplo inválido:

```
{
  "source": "auth",
  "message": "Failed login"
}
```

Falta `severity`.

source vacío

No se acepta:

```
{
  "source": "",
  "severity": 3,
  "message": "Test event"
}
```

Porque `source` tiene:

```
min_length=1
```

source demasiado largo

No se acepta un `source` de más de 64 caracteres.

Esto evita errores posteriores con la columna `Event.source`, definida como `String(64)`.

severity fuera de rango

No se aceptan severidades menores que 0 ni mayores que 10.

Ejemplos inválidos:

```
{
  "source": "auth",
  "severity": -1,
  "message": "Invalid severity"
}
```

```
{
  "source": "auth",
  "severity": 11,
  "message": "Invalid severity"
}
```

message vacío

No se acepta un mensaje vacío:

```
{
  "source": "auth",
  "severity": 3,
  "message": ""
}
```

Porque `message` tiene:

```
min_length=1
```

meta es opcional

El campo `meta` puede no enviarse.

Esto es válido:

```
{
  "source": "auth",
  "severity": 3,
  "message": "Login failed"
}
```

meta.host tiene importancia especial

Aunque `meta` es flexible, en `ingest.py` se usa concretamente:

```
ev.meta.get("host")
```

para calcular el `group_key`.

Por tanto, si se quiere que el evento pueda participar en lógica de agrupación, threshold o anti-duplicado por grupo, conviene incluir:

```
{
  "meta": {
    "host": "server-01"
  }
}
```

9 Comandos útiles relacionados

Probar un payload válido contra `/ingest` :

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 4,
  "message": "Failed login attempt for user admin",
  "meta": {
    "host": "server-01",
    "user": "admin",
    "ip": "192.168.1.10"
  }
}'
```

Probar un evento sin `meta` :

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 2,
  "message": "Simple event without metadata"
}'
```

Probar error por `severity` fuera de rango:

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 11,
  "message": "Invalid severity"
}'
```

Probar error por `source` vacío:

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "",
  "severity": 3,
  "message": "Invalid source"
}'
```

Comprobar el schema desde Swagger:

```
http://localhost:8000/docs
```

Comprobar que el schema se puede importar:

```
docker exec -it siem-api python -c "from app.schemas.ingest import IngestPayload; print(IngestPayload)"
```

Siguiente nota recomendada:

```
```text
04_API-de-ingesta
└─ 05_Analisis-tecnico-de-ingesta
 └─ 04_schema-event-py
```

Dataview (inline field 'Field(...)': Error:

-- PARSING FAILED -----

```
> 1 | Field(...)
 | ^
```

Expected one of the following:

'(', ')', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable

Dataview (for inline query '= Field(ge=0, le=10)': Unrecognized function name 'Field'

Dataview (for inline query '= Field(min\_length=1)': Unrecognized function name 'Field'